Aritmetica binaria e ALU

Reti Logiche T

Ingegneria Informatica

Numeri binari senza segno

- Come abbiamo visto, un numero senza segno N in una macchina digitale è rappresentato in **notazione posizionale in base 2**.
- Altra differenza fondamentale con la rappresentazione a cui siamo abituati «su carta» è che il numero di cifre disponibili è **fisso e finito:** l'hardware che uso stabilisce quanti segnali e quindi **bit** ho a disposizione per rappresentarlo.
- Dati n bit, i numeri senza segno **rappresentabili** sono compresi tra 0 e $2^n - 1$
- Esempio: in C, il tipo `unsigned short` ha (tipicamente) dimensione 16 bit, i numeri rappresentabili appartengono quindi all'intervallo $[0,65.535]$.
- La variabile `unsigned short a=62;` è rappresentato in memoria dai due byte $(00000000)_2$ $(00111110)_2$ ovvero $(003E)_{16}$ se scritti in modo compatto con notazione esadecimale.

Aritmetica binaria senza segno

- Somme e sottrazioni tra numeri binari si eseguono su carta esattamente come abbiamo imparato a fare con i numeri decimali alla scuola elementare
- Differenza fondamentale: con carta e penna posso sempre aggiungere cifre, nei sistemi digitali ho un numero **fisso e finito di cifre disponibili**
- Conseguenza: la somma di due numeri a n bit può **non** essere rappresentabile con il numero di bit disponibile (**overflow**)
- Esempio: con 6 cifre binarie posso rappresentare numeri senza segno compresi tra 0 e $63 (= 2^6 - 1)$. Se la somma è minore di 63 il risultato è valido, altrimenti ho overflow è il risultato **non è valido**
- La condizione di overflow è segnalata dall'**ultimo riporto** (**carry out**)

```
0 1          0 0 0 0 1 0
  2 6 +        0 1 1 0 1 0 +
  3 5          1 0 0 0 1 1
---------    -----------------
 (6 1)10      (1 1 1 1 0 1)2 =
              (6 1)10
```

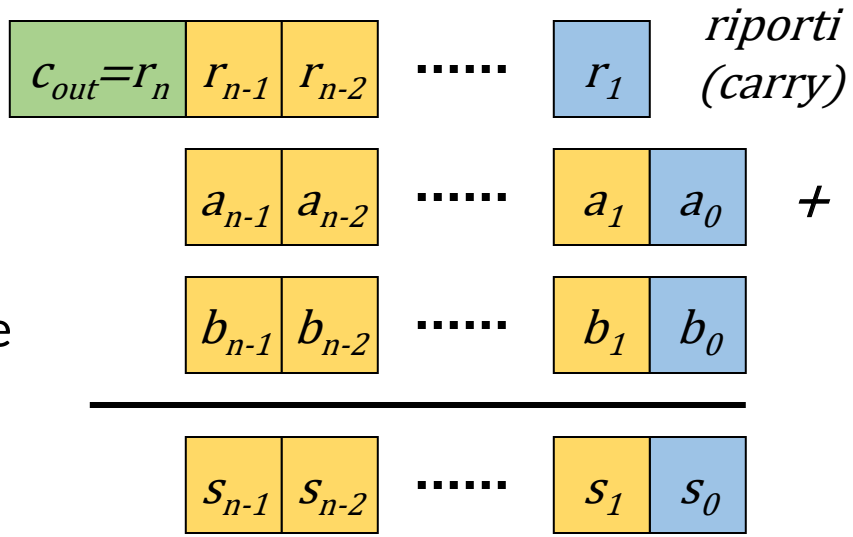
Carry out 0: OK, risultato rappresentabile a 6 bit

```
0 0          1 1 1 1 1 0
  5 4 +        1 1 0 1 1 0 +
  3 1          0 1 1 1 1 1
---------    -----------------
 (8 5)10      (0 1 0 1 0 1)2 =
              (2 1)10
```

Carry out 1: risultato NON valido, non rappresentabile a 6 bit

Adder

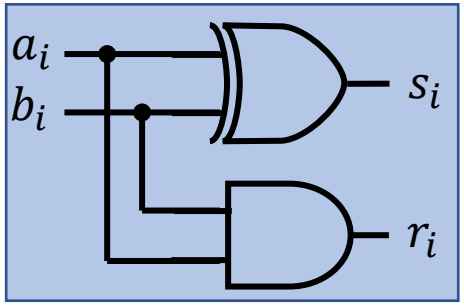
- Per definire la rete combinatoria che realizza la somma di due numeri A e B a n bit, avrei in teoria bisogno di una **tabella della verità con 2^{2n} righe** ($2n$ ingressi)
- Posso però usare un **approccio modulare** notando che devo ripetere le stesse operazioni per ogni bit: sommare i due bit di input a_i e b_i al riporto dei bit precedenti r_i ottenendo il bit di somma s_i e calcolare il nuovo riporto r_{i+1}
- Devo quindi solo definire una rete combinatoria che dati tre bit in input a_i b_i e r_i genera due bit di output: la loro somma s_i e il riporto r_{i+1} per i bit successivi
- La prima coppia di bit è un caso particolare in quanto non ho riporto e ho quindi un input in meno
- Oltre agli n bit che rappresentano la somma $s_0 \dots s_{n-1}$ in uscita avrò il valore dell'ultimo riporto r_n che rappresenta il carry out c_{out} dell'operazione complessiva

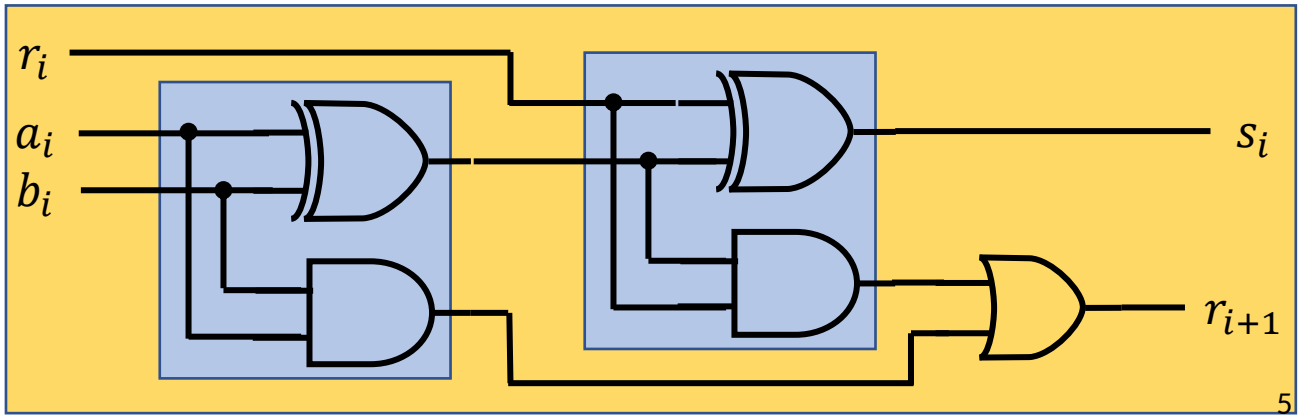


Full Adder e Half Adder

- La rete che realizza la somma di due bit nel caso generale con riporto viene detta **full adder.** Quando ho solo due bit in input invece si parla di **half adder**
- L'half adder è realizzabile con un EX-OR per la somma e un AND per il riporto, come si vede dalla parte in blu della tabella della verità di un full adder
- Il full adder è realizzabile a partire da due half adder e un OR: la somma di 3 bit è la somma dei primi due (EX-OR) che viene poi sommata al terzo (altro EX-OR); il riporto complessivo vale 1 se ho riporto nella prima somma OR nella seconda

r_i	a_i	b_i	s_i	r_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





Full Adder : da espressioni minime a struttura standard

r \ ab	00	01	11	10
0	0	0	1	0
1	0	1	1	1

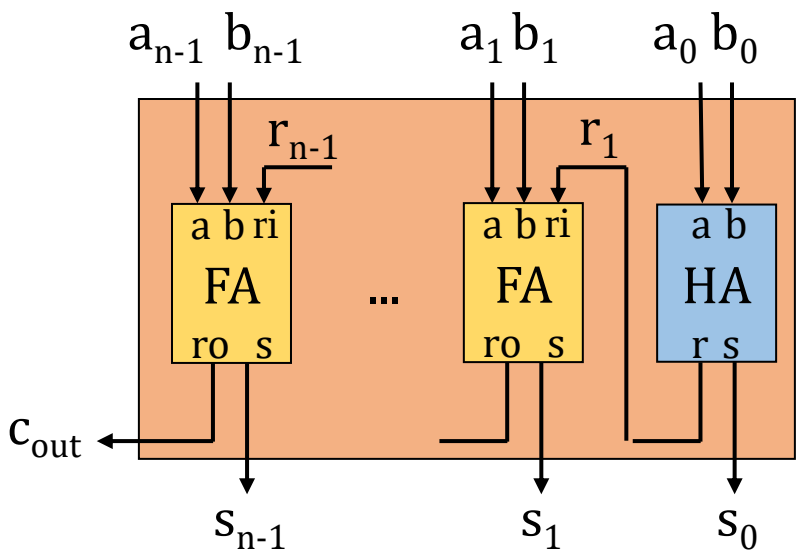
$$R = ab + br + ar$$
$$= ab + br \cdot 1 + a \cdot 1 \cdot r$$
$$= ab + (a + a')br + a(b + b')r$$
$$= ab + abr + a'br + abr + ab'r$$
$$= ab + abr + a'br + ab'r$$
$$= ab(r + 1) + (a'b + ab')r$$
$$= ab + (a \oplus b)r$$

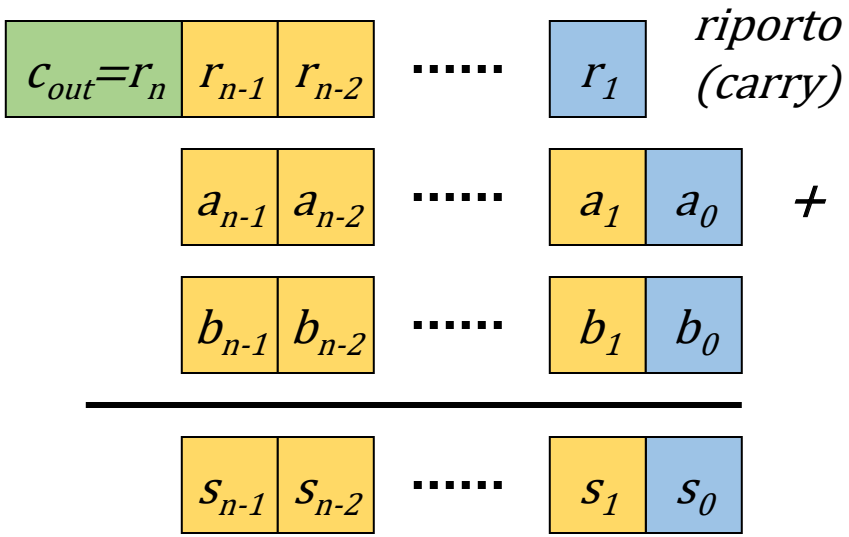
r \ ab	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$S = a'b'r + a'br' + abr + ab'r'$$
$$= r'(a'b + ab') + r(a'b' + ab)$$
$$= r'(a \oplus b) + r(a \oplus b)'$$
$$= r \oplus (a \oplus b)$$

Adder a n bit…

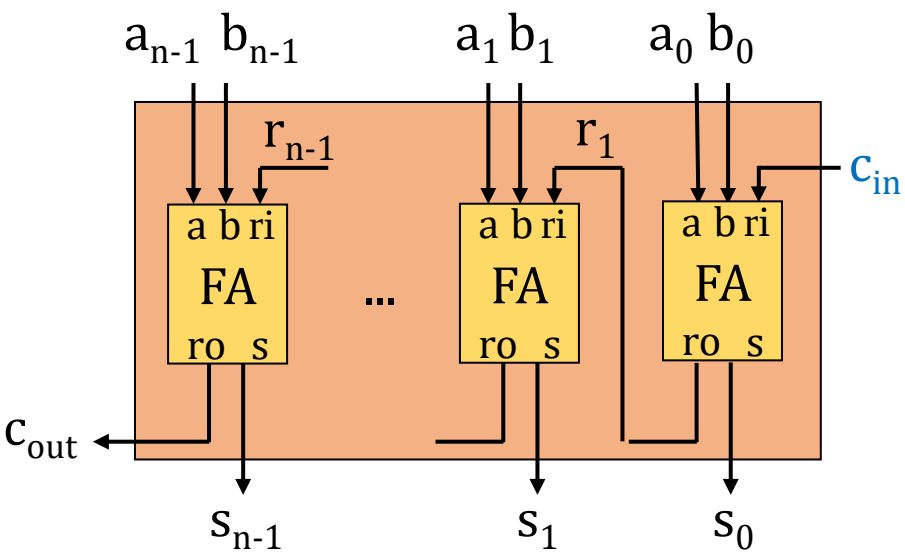
- Per realizzare un sommatore a n bit, è sufficiente disporre in serie un half adder e $n-1$ full adder. Ogni half o full adder realizza una delle colonne della somma in colonna.
- Il riporto dell'ultimo full adder rappresenta il bit di carry out $\mathbf{c_{out}}$ che indica la validità o meno del risultato.



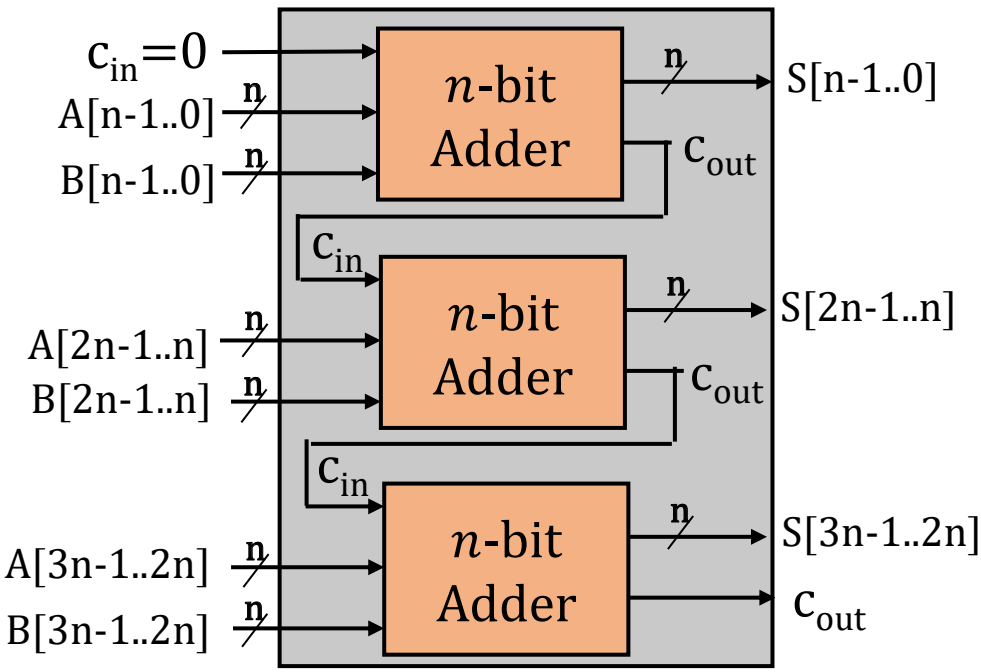


… e oltre

- Se si sostituisce l'unico half adder con un full adder si ottiene un sommatore con un ingresso aggiuntivo di **carry in c_{in}**, che può essere usato per disporre adder a n bit in serie e realizzare **sommatori con un numero di ingressi maggiore** di $2n$ (ovvero per sommare numeri con un numero di bit maggiori di n)



n-bit Adder



Adder a $3n$ bit

Complemento a 1

- Definiamo il **complemento a $\beta - 1$** di un numero in base β: dato un numero A **rappresentato da n cifre** il complemento a $\beta - 1$ di tale numero è $(\beta^n - 1) - \mathrm{A}$
- Es.: il complemento a $9(= 10 - 1)$ di $(001234)_{10}$ è
$$(10^6 - 1) - 001234 = 999999 - 001234 = 998765$$
- Es.: il complemento a $1(= 2 - 1)$ di $(001110)_2$ è
$$(2^6 - 1) - 001110 = 111111 - 001110 = 110001$$
- In generale, il complemento a $\beta - 1$ si ottiene per ogni cifra indipendentemente sottraendola da $\beta - 1$
- Nel caso del sistema binario, il **complemento a 1 equivale all'operatore NOT applicato ad ogni cifra**

Complemento a 2

- Definiamo il **complemento a $\boldsymbol{\beta}$** di un numero in base β: dato un numero A **rappresentato da $\boldsymbol{n}$ cifre** il complemento a β di tale numero è $\beta^n - \mathrm{A}$
- Possiamo calcolare il complemento a β sommando 1 al complemento a $\beta - 1$
- Nel caso binario, otteniamo anche una regola più semplice: il **complemento a 2** di un numero A **è dato da $\mathbf{NOT(A) + 1}$**
- Es.: il complemento a 2 di $(001110)_2$ è

$$2^6 - 001110 = 1000000 - 001110 = 110010$$

$$\text{o anche } \mathrm{NOT}(001110) + 1 = 110001 + 1 = 110010$$

- Un modo equivalente di calcolarlo è quello di lasciare invariato il numero fino al primo «1» partendo da destra incluso, e complementare tutte le altre cifre
- Se calcolo il complemento a 2 di un complemento a 2 ottengo il numero di partenza: $00110100 \rightarrow 11001100 \rightarrow 00110100$

Complemento a 2

A cosa serve la rappresentazione in complemento a 2?

1. **A implementare la sottrazione come somma**, quindi ad usare lo stesso semplice ed efficiente circuito che abbiamo già sintetizzato per eseguire somme e sottrazioni
2. **A rappresentare i numeri con segno quando sono negativi** in maniera tale che le operazioni su di essi possano essere eseguite in modo efficiente

Sottrazione tra numeri senza segno

- Dati due numeri binari senza segno a n bit A e B con $\mathrm{A} > \mathrm{B}$, si può calcolare $\mathrm{A} - \mathrm{B}$ sommando A al complemento a 2 di B e ignorando il carry out uguale a 1.
- Infatti, il complemento a 2 di B è il numero $2^n - \mathrm{B}$, quindi il risultato della somma è $\mathrm{A} + (2^n - \mathrm{B}) = (\mathrm{A} - \mathrm{B}) + 2^n$

$$
\begin{array}{r}
(44)_{10} \; - \\
(26)_{10} \; = \\
\hline
(18)_{10}
\end{array}
$$

$$
\begin{array}{r}
(101100)_2 \; - \\
(011010)_2 \; = \\
\hline
(010010)_2
\end{array}
$$

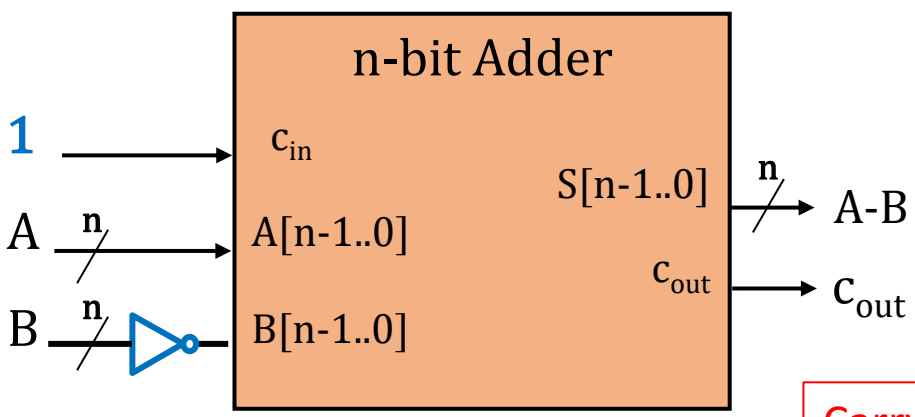
$44 + \text{comp. a 2 di } 26$

$$
\begin{array}{rrrrrrl}
\mathbf{1} & 0 & 1 & 1 & 0 & 0 & \\
 & 1 & 0 & 1 & 1 & 0 & 0 \; + \\
 & 1 & 0 & 0 & 1 & 1 & 0 \\
\hline
 & (0 & 1 & 0 & 0 & 1 & 0)_2 \; = \\
 & (1 & 8)_{10} & & & &
\end{array}
$$

Carry out 1: OK, risultato rappresentabile a 6 bit

Sottrazione tra numeri senza segno

- Poiché il complemento a 2 si ottiene come $\mathrm{NOT}(B) + 1$, Per realizzare la sottrazione $A - B$ come somma di A con il complemento a 2 di B, è sufficiente usare un adder a n bit con
  - **B negato in ingresso all'adder**
  - **e $c_{in} = 1$**
- Se $(\mathrm{A} - \mathrm{B}) < 0$, il risultato non è rappresentabile come numero senza segno e il risultato della somma, pari a $(\mathrm{A} - \mathrm{B}) + 2^n$ è minore di 2^n, ovvero carry out vale «0». Quindi, **nella sottrazione, vale la regola contraria della somma**: carry out =1 indica operazione eseguita correttamente, carry out = 0 indica overflow.



26 + comp. a 2 di 44

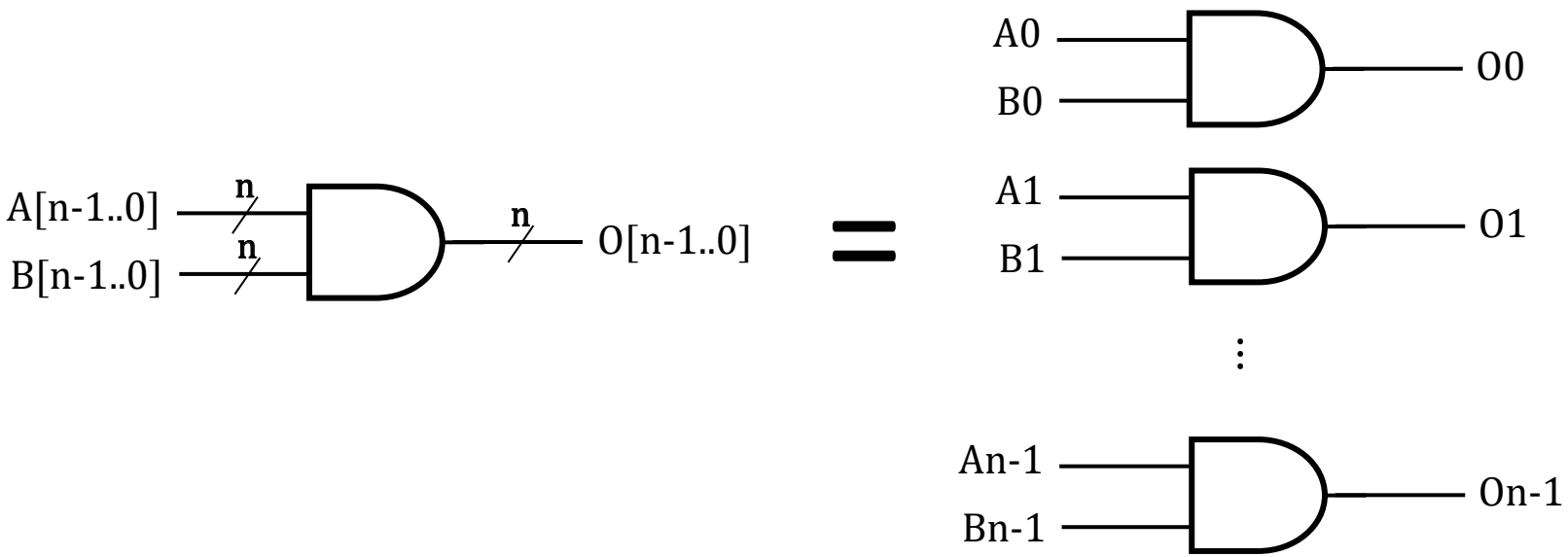
```
0 1 0 0 0 0
  0 1 1 0 1 0 +
  0 1 0 1 0 0
-----------------
 (1 0 1 1 1 0)2 =
 (4 6)10
```

Carry out 0: risultato NON rappresentabile a 6 bit

$(26-44)+2^6$
$=-18+64=46$

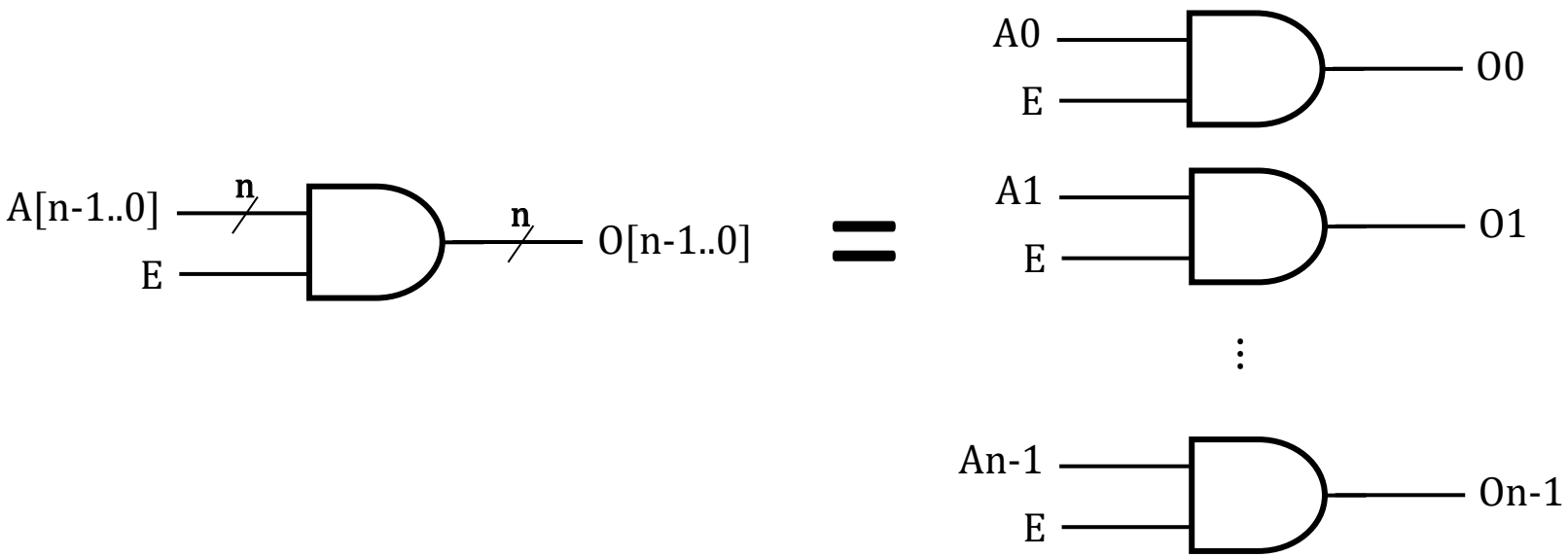
Notazione

- Per indicare gate o reti combinatorie che operano su bus, possiamo disegnare un unico gate. Quando portiamo un bus su un ingresso che normalmente opera su un singolo bit, nel circuito reale saranno presenti n gate o reti in parallelo, se il parallelismo del bus è n.



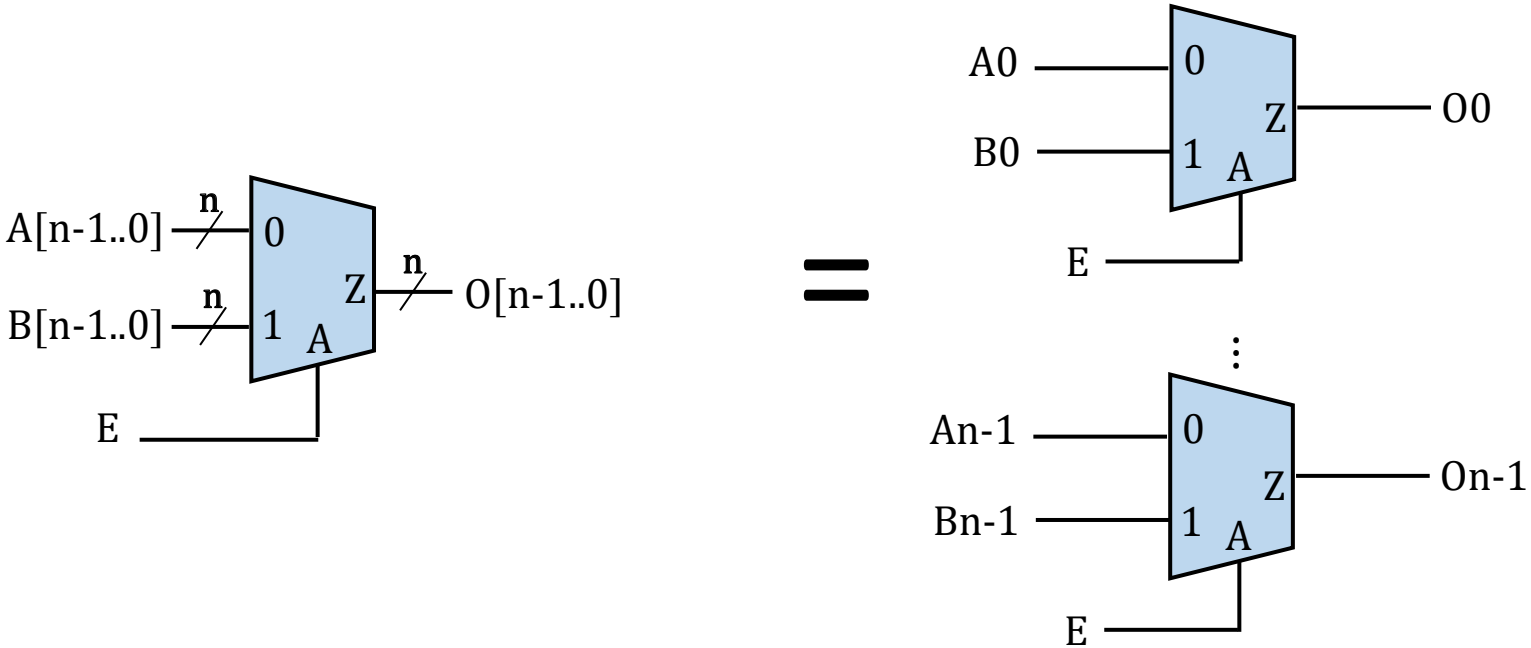
Notazione

- Se uno degli ingressi è un singolo bit e l'altro è un bus, la struttura risultante avrà ancora n gate o reti in parallelo, con il bit singolo ripetuto in tutti gli n gate o reti. **L'uscita è ancora un bus con parallelismo n!**



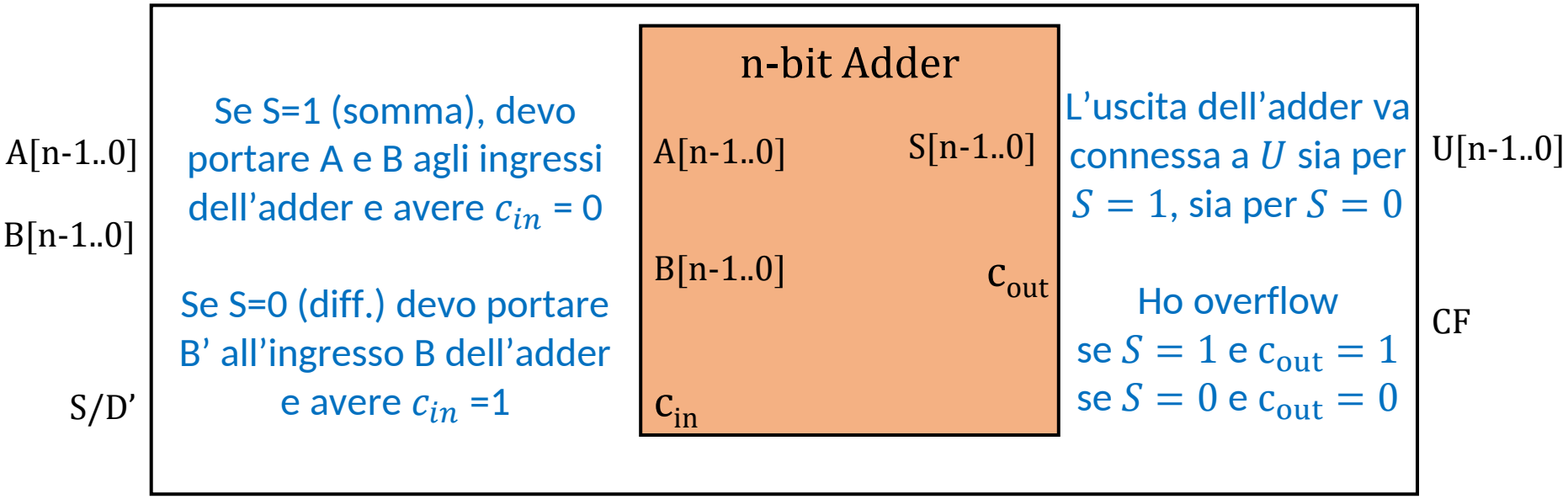
Notazione

- Se uno degli ingressi è un singolo bit e l'altro è un bus, la struttura risultante avrà ancora n gate o reti in parallelo, con il bit singolo ripetuto in tutti gli n gate o reti. **L'uscita è ancora un bus con parallelismo n.**



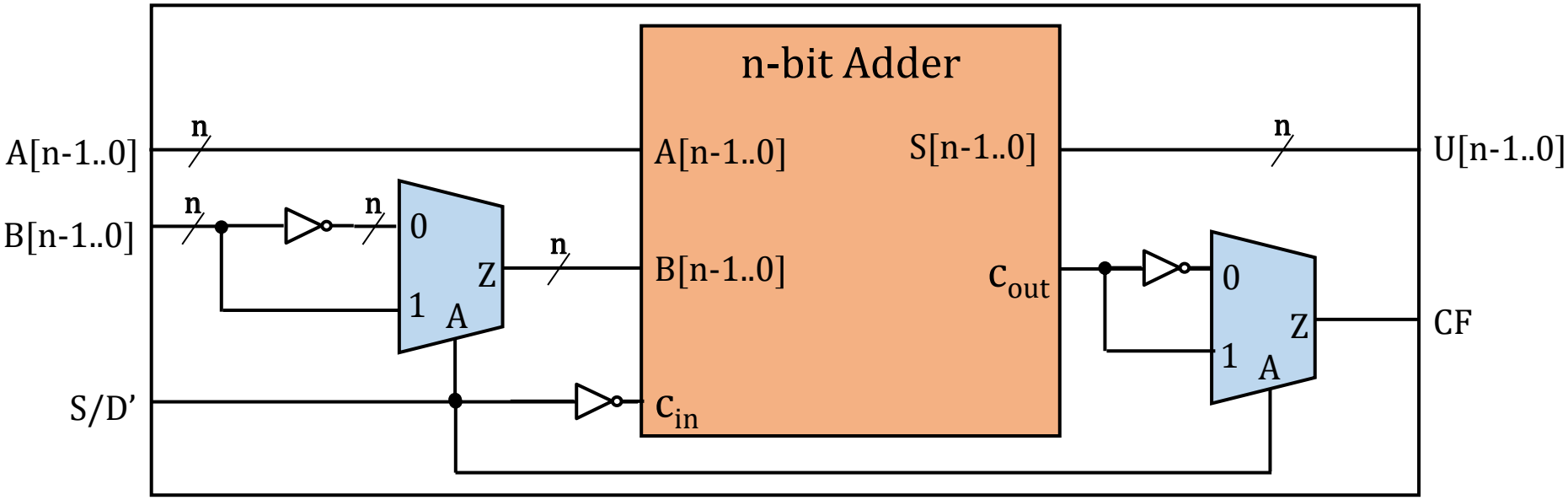
Esercizio: Adder/Subtracter

- Come realizzo un circuito comandato da un bit di input S/D' che: quando $S = 1$ presenta sulle n uscite $U[n-1..0]$ la somma dei due numeri in input $A[n-1..0]$ e $B[n-1..0]$ a n bit; quando $S = 0$ riporta la sottrazione $A - B$; e in entrambi i casi asserisce il bit di uscita CF(carry flag) se si è verificato overflow nello svolgimento dell'operazione? Nota: le uscite che indicano il verificarsi di condizioni particolari nel risultato prendono il nome di **flag**.



Esercizio: Adder/Subtracter

- Come realizzo un circuito comandato da un bit di input S/D' che: quando $S = 1$ presenta sulle n uscite $U[n-1..0]$ la somma dei due numeri in input $A[n-1..0]$ e $B[n-1..0]$ a n bit; quando $S = 0$ riporta la sottrazione $A - B$; e in entrambi i casi asserisce il bit di uscita CF(carry flag) se si è verificato overflow nello svolgimento dell'operazione? Nota: le uscite che indicano il verificarsi di condizioni particolari nel risultato prendono il nome di **flag**.



Numeri con segno: segno e valore assoluto

- Sono possibili **più rappresentazioni** per i numeri con segno
- La più immediata, <u>ma NON quella usata nelle reti logiche</u>, è la rappresentazione in **segno e valore assoluto**
- Data la stringa che rappresenta un numero intero con segno $(\mathrm{A})_2 = a_{n-1}a_{n-2} \dots a_0$
  - a_{n-1} rappresenta il segno (0:+, 1:-)
  - $a_{n-2} \dots a_0$ il valore assoluto del numero
- I numeri rappresentabili appartengono all'intervallo
$$-(2^{n-1} - 1) \leq A \leq 2^{n-1} - 1$$
- Esempio: con n=4 cifre posso rappresentare i numeri da -7 a 7
- Ho due rappresentazioni possibili per 0

$(\mathrm{A})_{10}$	a_3	a_2	a_1	a_0
+7	0	1	1	1
+6	0	1	1	0
+5	0	1	0	1
+4	0	1	0	0
+3	0	0	1	1
+2	0	0	1	0
+1	0	0	0	1
+0	0	0	0	0
-0	1	0	0	0
-1	1	0	0	1
-2	1	0	1	0
-3	1	0	1	1
-4	1	1	0	0
-5	1	1	0	1
-6	1	1	1	0
-7	1	1	1	1

Numeri con segno: complemento a 2

- La <u>rappresentazione usata</u> è quella in **complemento a 2**.
- La stringa che rappresenta un numero intero con segno $(\mathrm{A})_2 = a_{n-1}a_{n-2} \dots a_0$ si ottiene
  - Se A è positivo, come per segno e valore assoluto
  - Se A è negativo, come il complemento a 2 del numero positivo $-\mathrm{A}$, bit di segno incluso
- Il bit più significativo codifica quindi (implicitamente) il segno, come nella rappresentazione segno e valore assoluto
- I numeri rappresentabili appartengono all'intervallo (asimmetrico)

$$-2^{n-1} \leq A \leq 2^{n-1} - 1$$

- Un solo modo di rappresentare 0
- Perché è usata? Perché **operazioni elementari tra numeri rappresentati in complemento a 2 danno come risultato ancora numeri in complemento a 2**, mentre questo non è vero per la rappresentazione in segno e valore assoluto

$(\mathrm{A})_{10}$	a_3	a_2	a_1	a_0
+7	0	1	1	1
+6	0	1	1	0
+5	0	1	0	1
+4	0	1	0	0
+3	0	0	1	1
+2	0	0	1	0
+1	0	0	0	1
+0	0	0	0	0
-1	1	1	1	1
-2	1	1	1	0
-3	1	1	0	1
-4	1	1	0	0
-5	1	0	1	1
-6	1	0	1	0
-7	1	0	0	1
-8	1	0	0	0

Valore ed estensione di un numero in complemento a 2

- Per **calcolare il valore** di un numero in complemento a 2 (ovvero per convertirlo in base 10) si può usare la solita formula della notazione posizionale, in cui **il bit più significativo viene moltiplicato per il valore negativo corrispondente a quella posizione**.
- Esempio: se la stringa $(11100)_2$ rappresenta un numero in complemento a 2, il valore corrispondente è

$$1 * -\mathbf{2^4} + 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 0 * 2^0 = -16 + 8 + 4 = (-4)_{10}$$

- Ne segue che per **estendere il numero di bit** di un numero rappresentato in complemento a 2, **si deve replicare il bit più significativo**. Per esempio, per passare da 5 a 7 bit,

$$(11100)_2 = -4 \rightarrow (\mathbf{111}1100)_2 = (-64 + 32 + 16) + 8 + 4 =$$
$$= -16 + 8 + 4 = (-4)_{10}$$

$$(01100)_2 = 8 + 4 = (12)_{10} \rightarrow (\mathbf{00}01100)_2 = 8 + 4 = (12)_{10}$$

Bit e informazione

- **IMPORTANTE**: una stringa di bit è sempre e solo una stringa di bit
- L'interpretazione è «negli occhi di chi guarda»
- Es: la stringa 1111, se interpretata come
  - numero binario senza segno rappresenta il numero $(15)_{10}$
  - numero binario con segno rappresentato in segno e valore assoluto, rappresenta il numero $(-7)_{10}$
  - numero binario con segno rappresentato in complemento a 2, rappresenta il numero $(-1)_{10}$
- Vale per i bit in una macchina digitale quello che vale per i file binari in un calcolatore: chi «trasmette» e chi «riceve» l'informazione devono essersi accordati sul suo formato perché questa possa essere recuperata a partire dalle stringhe di bit

Aritmetica binaria con segno

- Anche per i numeri con segno la somma di due numeri a n bit può **non** essere rappresentabile con il numero di bit disponibile (**overflow**)
- Esempio: con 6 cifre binarie posso rappresentare numeri con segno compresi tra $-32 = (2^5)$ e $31 (= 2^5 - 1)$. Cosa mi segnala se esco da questo intervallo?
- La condizione di overflow **non è più segnalata dal carry out**, come si può vedere negli esempi qui sotto

```
0 0          1 1 1 1 1 0
 -1 0 +        1 1 0 1 1 0 +
 -1 8          1 0 1 1 1 0
---------    -----------------
(-2 8)10      (1 0 0 1 0 0)2 =
              (-28)10
```

Carry out 1 ma risultato rappresentabile a 6 bit

```
0 0          0 1 1 1 1 0
  3 0 +        0 1 1 1 1 0+
  0 6          0 0 0 1 1 0
---------    -----------------
 (3 6)10      (1 0 0 1 0 0)2 =
              (-28)10
```

Carry out 0 ma risultato NON valido, non rappresentabile a 6 bit

Aritmetica binaria con segno

- Per i numeri con segno la situazione di overflow è individuabile controllando **se gli operandi hanno segno concorde e diverso da quello del risultato** (ovvero, per esempio, sommo due numeri positivi e ottengo un numero negativo), che equivale a verificare se

$$a_{n-1} = b_{n-1} \text{ AND } b_{n-1} \neq s_{n-1}$$

- In realtà è sufficiente verificare se $r_{n-1} \neq c_{out}$ in quanto, se $a_{n-1} = b_{n-1}$ allora $s_{n-1} = r_{n-1}$ e $c_{out} \doteq r_n = b_{n-1}$

```
0 0          1 1 1 1 1 0
 -1 0 +        1 1 0 1 1 0 +
 -1 8          1 0 1 1 1 0
---------    -----------------
(-2 8)10      (1 0 0 1 0 0)2 =
              (-28)10
```

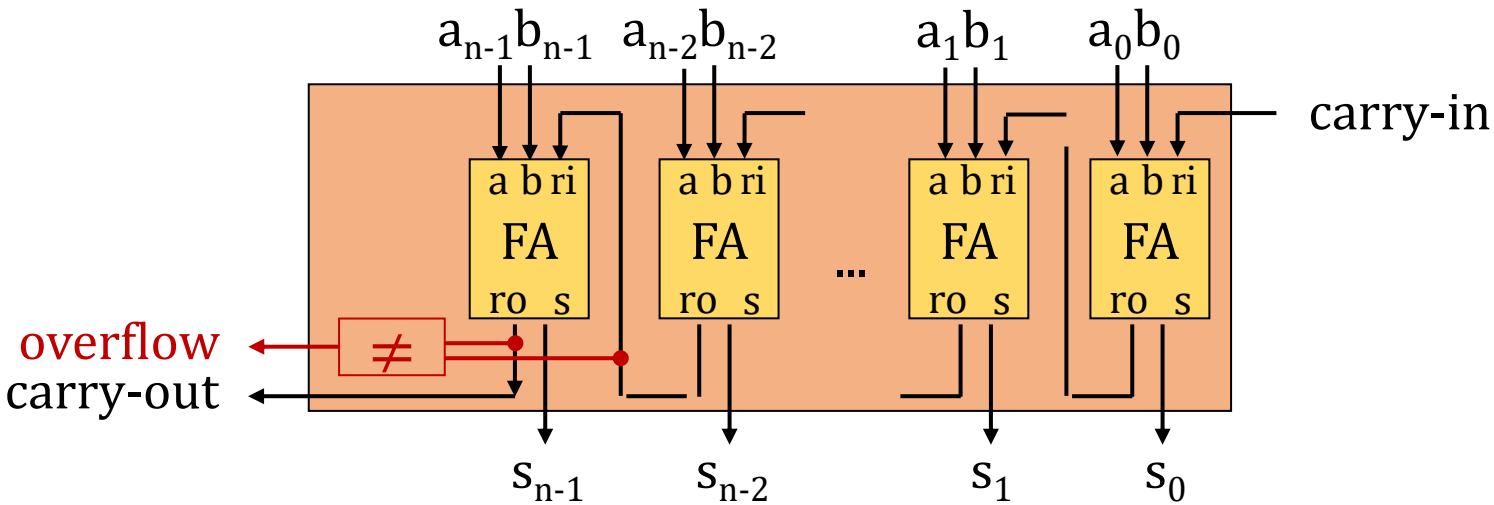
$c_{out} = r_{n-1}$: OK, risultato rappresentabile a 6 bit

```
0 0          0 1 1 1 1 0
  3 0 +        0 1 1 1 1 0+
  0 6          0 0 0 1 1 0
---------    -----------------
 (3 6)10      (1 0 0 1 0 0)2 =
              (-28)10
```

$c_{out} \neq r_{n-1}$: risultato NON valido, non rappresentabile a 6 bit

N bit adder modificato

- Per gestire numeri con segno, quindi, aggiungiamo un segnale di uscita di **overflow** all'adder a n bit che avevamo definito

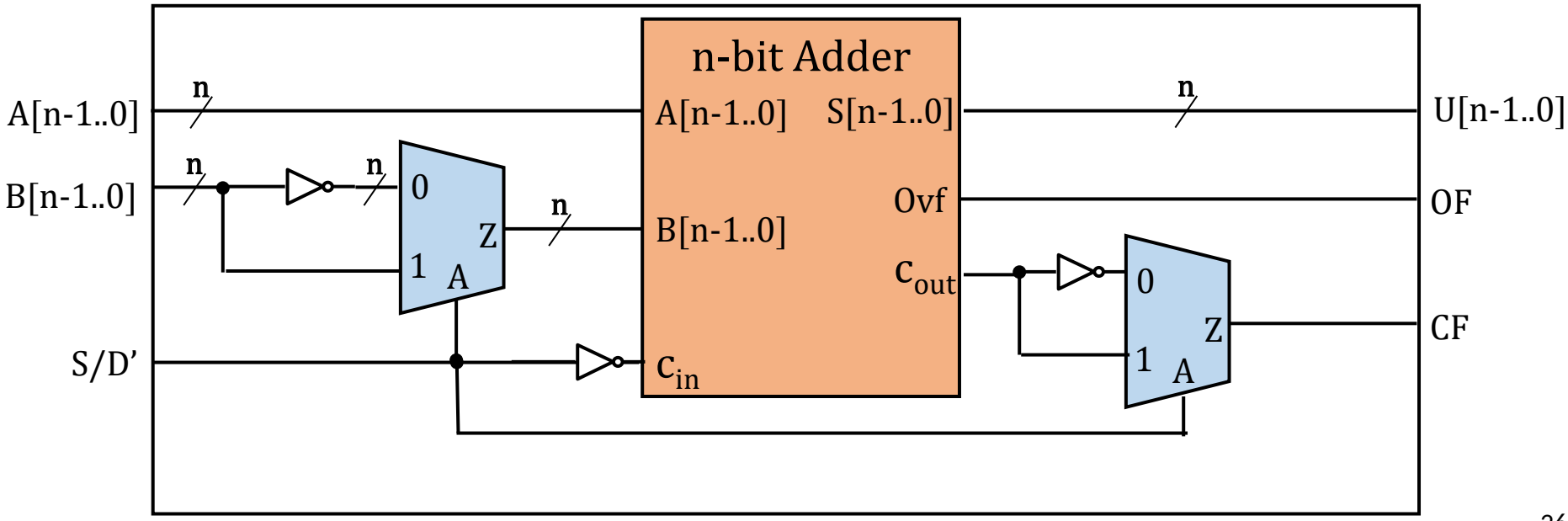


Sottrazione di numeri con segno

- Infine, come per i numeri senza segno, per sottrarre due numeri con segno, è sufficiente eseguire il complemento a due del sottraendo e sommarli. Infatti, eseguendo il complemento a 2 si cambia segno al numero e quindi la somma esegue la sottrazione voluta

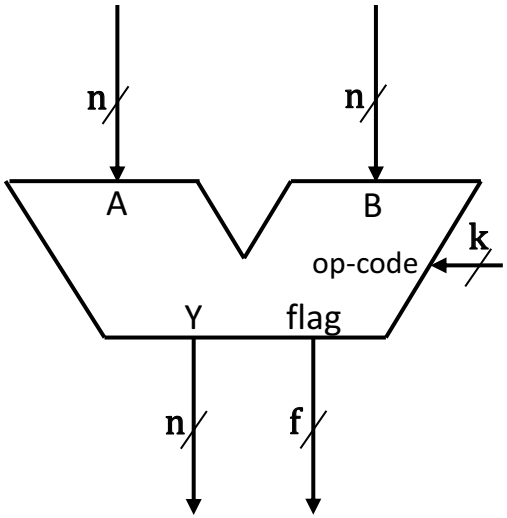
$$(\pm A) - (\pm B) = (\pm A) + (\mp B)$$

- Per ottenere un generico circuito che esegue somme e sottrazioni tra numeri con e senza segno, quindi, è possibile usare quello visto in precedenza. La condizione di overflow sarà segnalata dall'uscita CF (carry flag) o da OF (overflow flag) a seconda che sia stata eseguita un'operazione tra numeri con segno o senza.



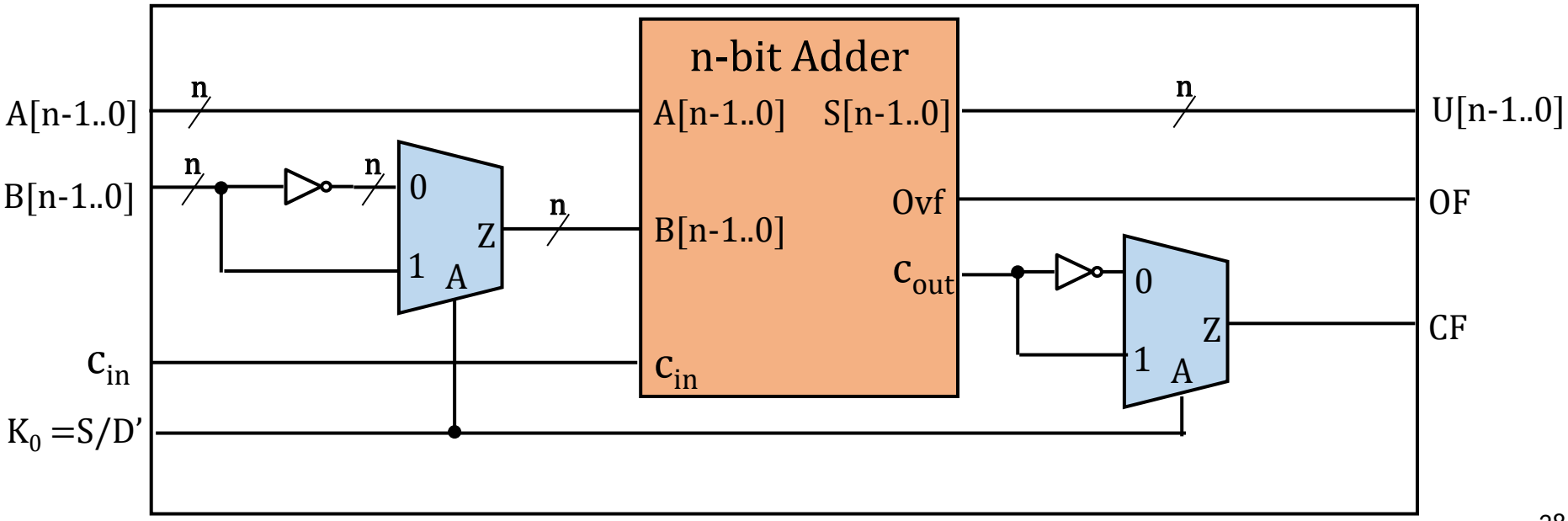
Arithmetic Logic Unit (ALU)

- La codifica dei numeri con e senza segno e i circuiti che abbiamo visto sono la base per costruire le cosiddette **Arithmetic Logic Unit (ALU)**.
- Le ALU sono macchine combinatorie in grado di eseguire operazioni aritmetiche (somma, sottrazione, complemento a 2, scorrimento a destra o sinistra, ...) e logiche (bit per bit AND, OR, XOR, ...) sui **due operandi** in base al **codice operativo** (op-code) in ingresso. Oltre a fornire il **risultato** dell'operazione, indicano anche attraverso una serie di **flag** il verificarsi di particolari condizioni nel risultato stesso.
- Le ALU sono il «motore» di un microprocessore e le CPU moderne hanno anche più di una ALU per core.



ALU – Operazioni aritmetiche

- Partiamo dal circuito precedente. Uniche differenze:
  - lasciamo gestire il bit di carry-in come un ulteriore ingresso, per permettere il montaggio in cascata di ALU
  - Indichiamo il bit S/D' come K_0, ovvero il bit meno significativo dell'op-code



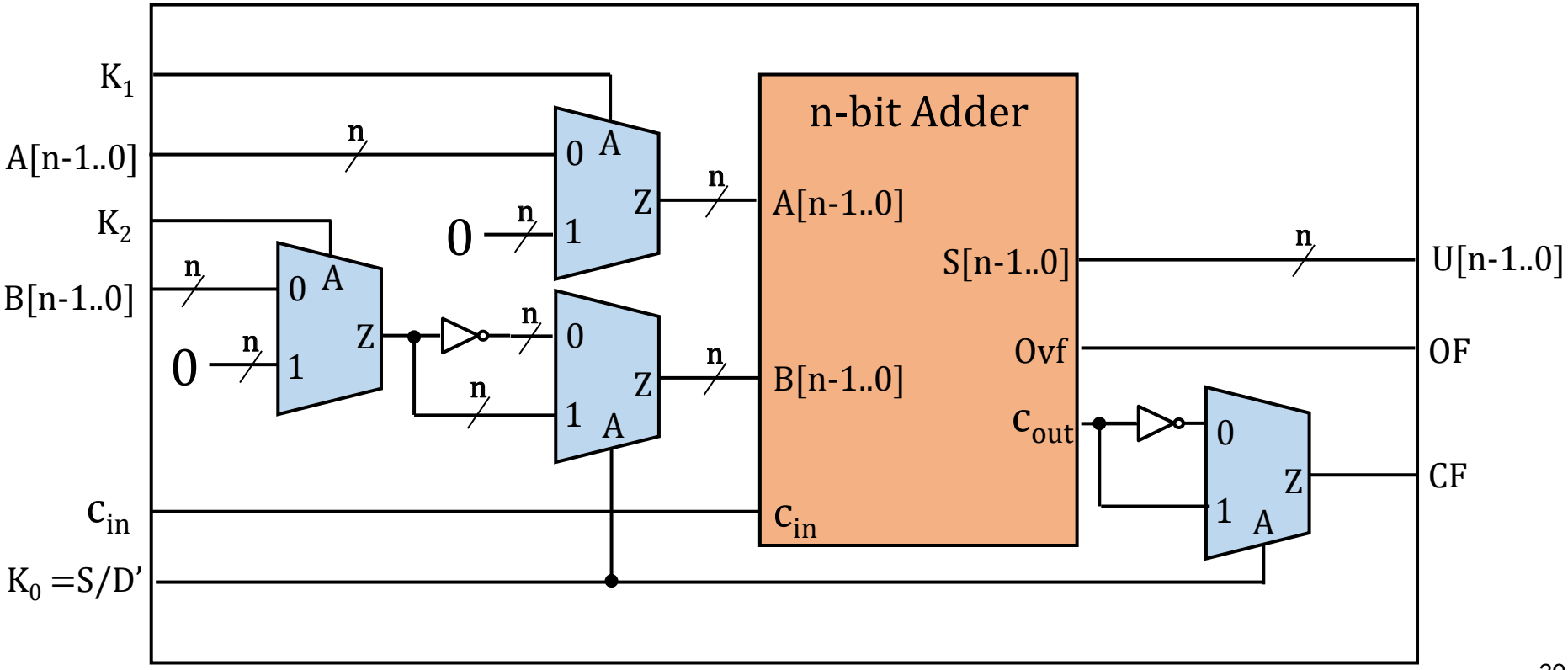
ALU – Operazioni desiderate

- Per mostrare, tramite un esempio, come si potrebbe progettare una semplice ALU, immaginiamo di voler aggiungere le seguenti **operazioni aritmetiche**
  - calcolo di $-B$, ovvero dell'opposto dell'ingresso B
  - operazione $A++$, ovvero produrre in uscita $A+1$
  - operazione $A--$, ovvero produrre in uscita $A-1$
  - produrre in uscita le costanti $0, 1, 2^n - 1$
  - continuando a rendere possibile il calcolo di $A+B$ e $A-B$

  E le seguenti **operazioni logiche**
  - NOT bit a bit di A oppure di B
  - AND, OR, XOR e EQUIVALENCE bit a bit tra A e B

ALU – Operazioni aritmetiche

- Per realizzare le operazioni, aggiungiamo due MUX sugli operandi che permettano di decidere se usare il valore in ingresso oppure la costante 0 a n bit tramite due ulteriori bit di op-code K_1 e K_2



Operazioni implementate

- La tabella seguente indica quali operazioni algebriche sono ottenibili al variare degli op-code.

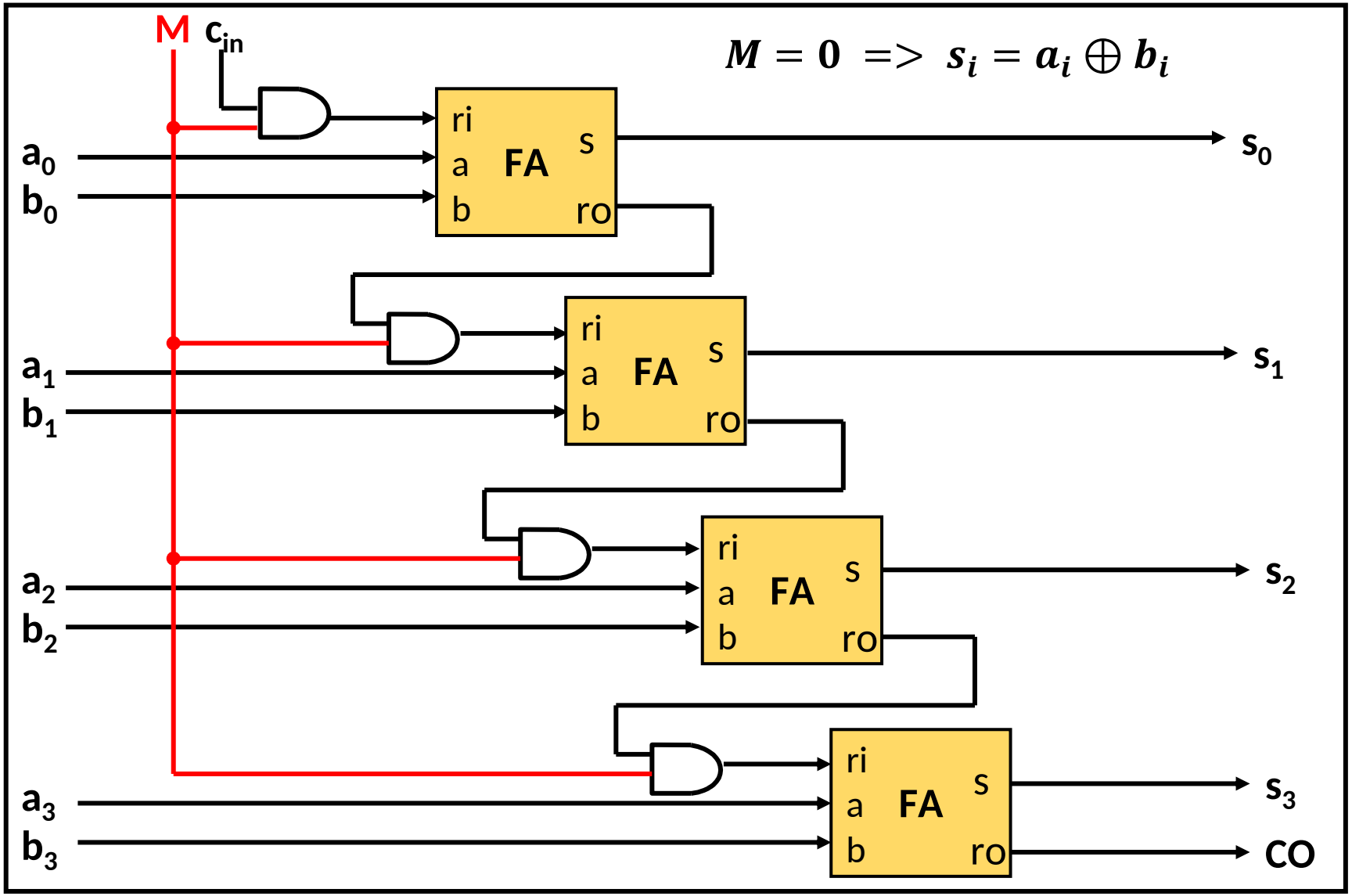
op-code $K_2K_1K_0$	$C_{in} = 0$	$C_{in} = 1$
000	$A - B - 1$	$A - B$
001	$A + B$	$A + B + 1$
010	$-B - 1$	$-B$
011	B	$B + 1$
100	$A - 1$	A
101	A	$A + 1$
110	$2^n - 1, -1$	0
111	0	1

Qui la ALU esegue
000…0 + 111…1 = 111…1
Se interpretato come numero senza segno il risultato vale $2^n - 1$, con segno vale -1

ALU – Operazioni logiche

- Aggiungiamo un bit di modo M che stabilisce se la ALU debba realizzare operazioni aritmetiche ($M = 1$) o logiche ($M = 0$)
- Le operazioni logiche, a differenza di quelle aritmetiche, **non prevedono l'uso di riporti:** ogni coppia di bit dei due operandi viene processata indipendentemente
- E' necessario quindi modificare l'adder a n bit che abbiamo usato fino ad ora per accettare l'ulteriore input M che abilita o meno la propagazione dei riporti: è sufficiente aggiungere l'AND del riporto con M in ingresso ad ogni full adder, in modo che il riporto sia sempre 0 quando l'adder opera in modo logico.

Adder a 4 bit con modo logico



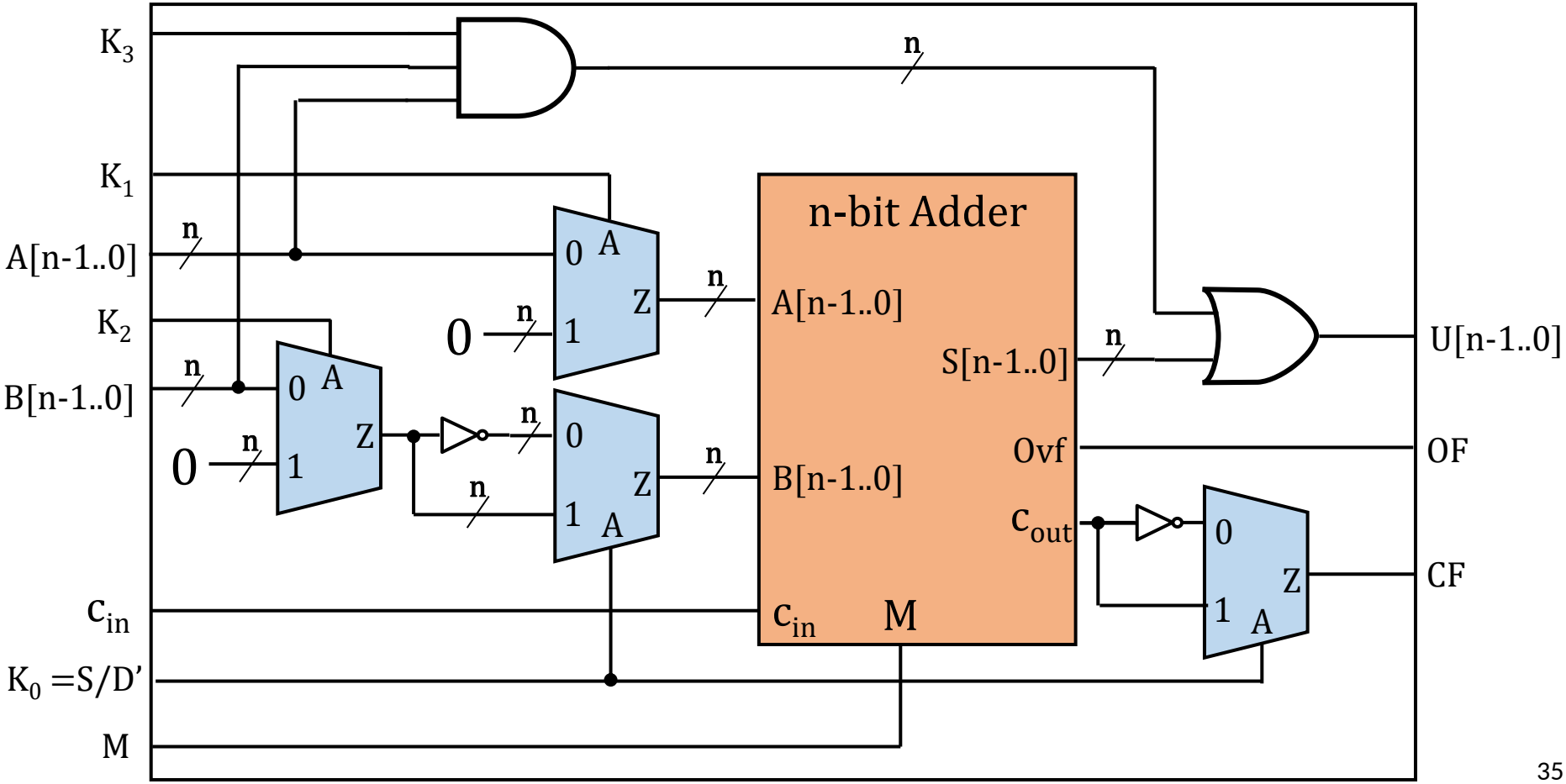
Operazioni logiche implementate

- La tabella seguente indica quali operazioni logiche sono ottenibili al variare degli op-code con $M = 0$.
- Il bit di carry in è ininfluente in modo logico
- Mancano le operazioni di AND e OR

op-code $K_2K_1K_0$	$C_{in} = -$
000	$A \oplus \bar{B} = A \equiv B$
001	$A \oplus B$
010	$0 \oplus \bar{B} = \bar{B}$
011	$0 \oplus B = B$
100	$A \oplus 1 = \bar{A}$
101	$A \oplus 0 = A$
110	$0 \oplus 1 = 1$
111	$0 \oplus 0 = 0$

ALU – AND e OR

- Aggiungiamo un AND tra ogni coppia di bit corrispondenti degli operandi la cui uscita viene poi posta in OR con l'uscita corrispondente dell'Adder. Un ulteriore bit di op-code K_3 consente di abilitare questa sottorete (con $K_3 = 0$ si ottengono tutti le uscite aritmetiche e logiche già viste).



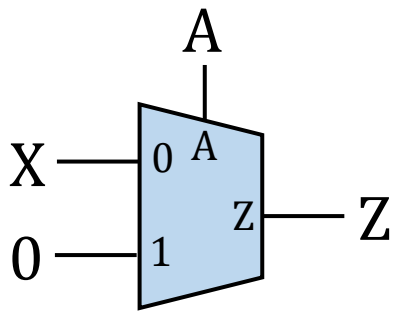
Operazioni logiche implementate

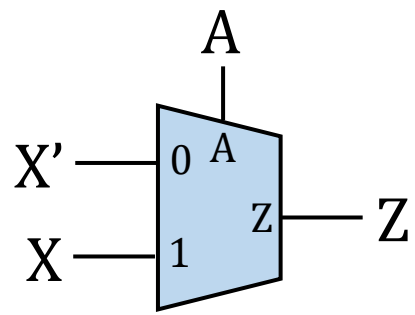
- La tabella seguente indica quali operazioni logiche sono ottenibili al variare degli op-code con $M = 0$.
- Il bit di carry in è ininfluente in modo logico
- In modo aritmetico, ovvero con $M = 1$, K_3 è mantenuto sempre a 0
- N.B.: i simboli in questa tabella si riferiscono a prodotto e somma **logici**

op-code $K_2K_1K_0$	$C_{in} = -$ $K_3 = 0$	$C_{in} = -$ $K_3 = 1$
000	$A \oplus \bar{B} = A \equiv B$	$A \equiv B$
001	$A \oplus B$	$A + B$
010	$0 \oplus \bar{B} = \bar{B}$	$A + \bar{B}$
011	$0 \oplus B = B$	B
100	$A \oplus 1 = \bar{A}$	$\bar{A} + B$
101	$A \oplus 0 = A$	A
110	$0 \oplus 1 = 1$	1
111	$0 \oplus 0 = 0$	AB

Semplificazioni

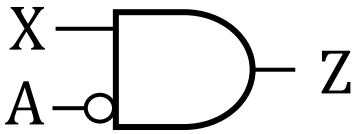
- MUX in cui le vie d'ingresso sono costanti o presentano lo stesso segnale in forma vera e negata possono essere sostituiti da reti più semplici, come si vede dalle espressioni SP del MUX

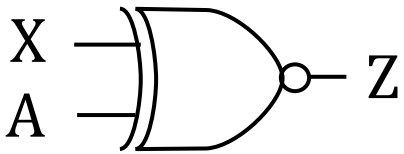




$$Z = A' \cdot X + A \cdot 0 = \mathrm{A'X}$$

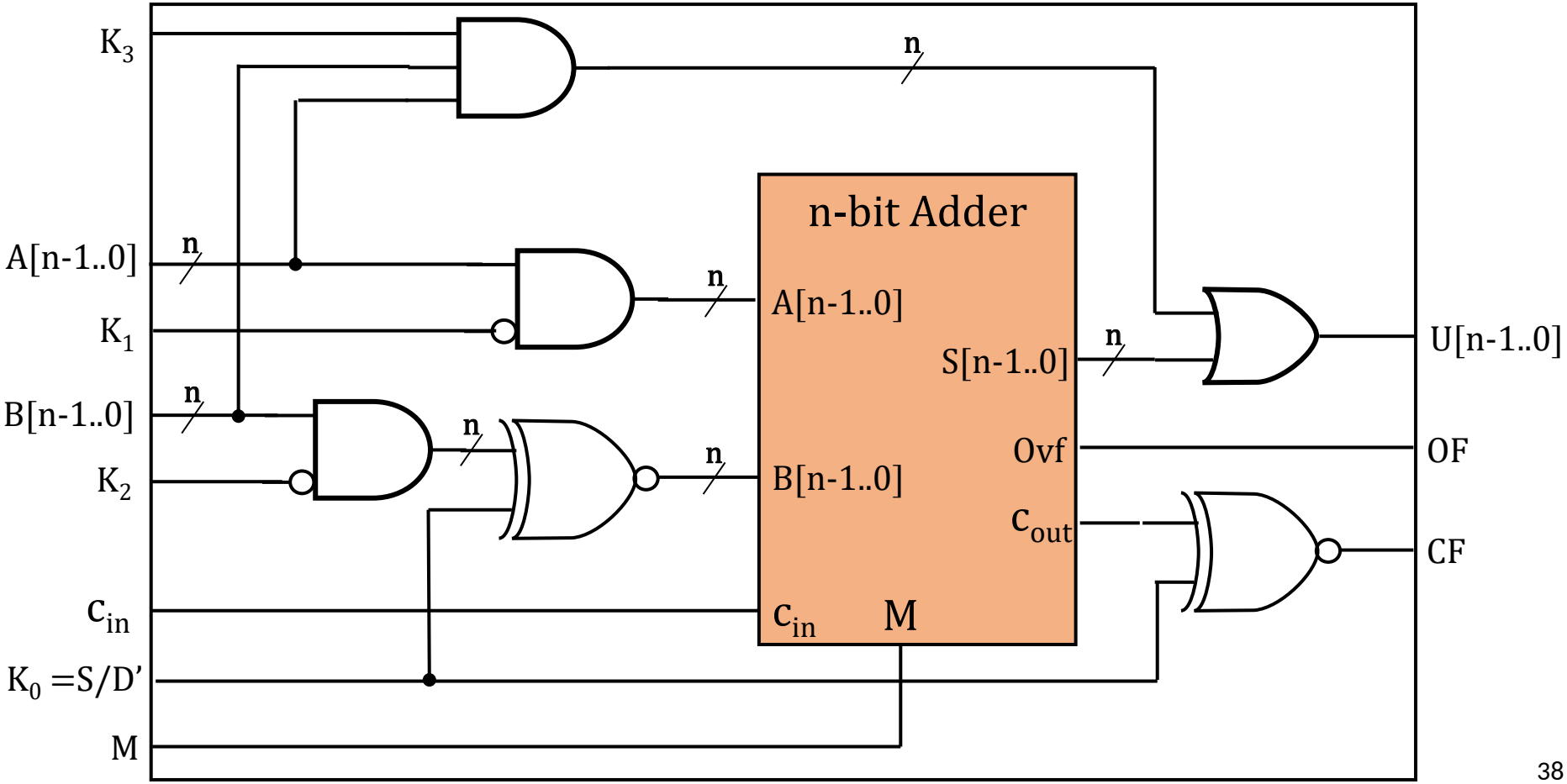
$$Z = A' \cdot X' + A \cdot X = (\mathrm{A} \equiv \mathrm{X})$$





ALU – Circuito semplificato

- Sfruttando le equivalenze appena viste è possibile arrivare al seguente circuito combinatorio per la ALU che abbiamo definito

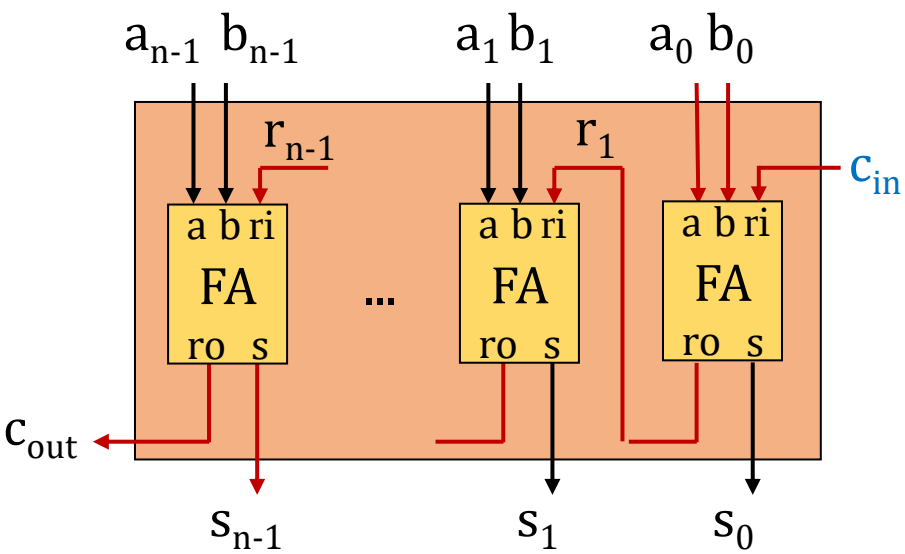


ALU – Ulteriori flag

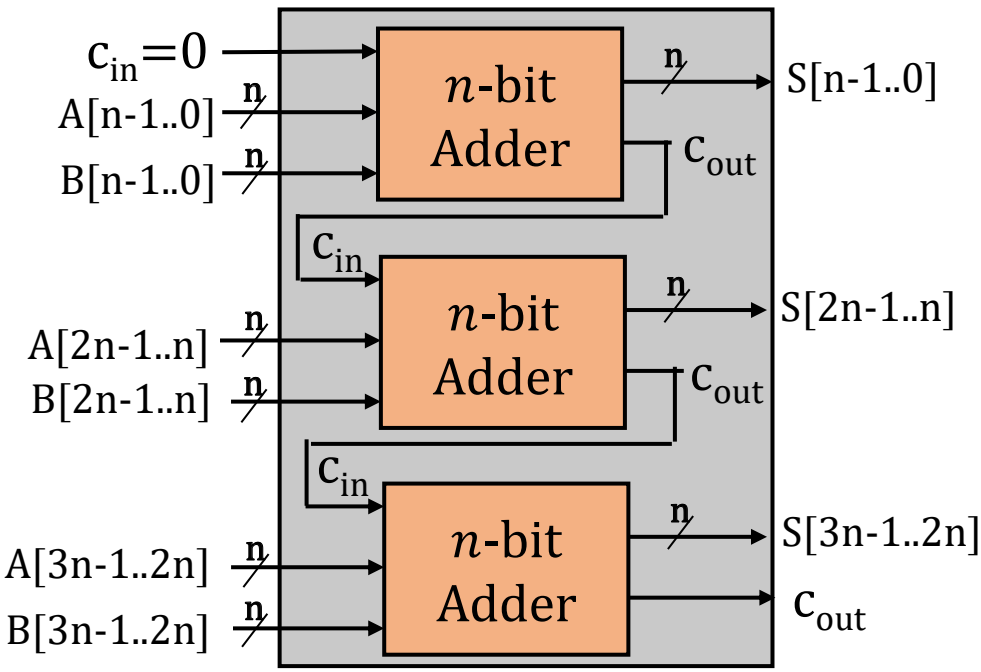
- E' utile fornire ulteriori indicazioni sul risultato tramite flag aggiuntivi oltre a quelli di overflow
- Aggiungiamo altri due flag alla nostra ALU:
  - Zero Flag (ZF): segnala se il risultato è zero.
  - Sign Flag (SF): segnala se il risultato è negativo (quando interpretato come numero rappresentato in complemento a 2)
- Possiamo ottenerli facilmente a partire dal risultato $\mathrm{S}[n-1..0]$ calcolato dall'adder
  - $ZF = (S_{n-1} + S_{n-2} + \cdots + S_0)' = S_{n-1} \downarrow S_{n-2} \downarrow \cdots \downarrow S_0$
  - $SF = S_{n-1}$

Adder - ritardo

Il limite principale dell'adder realizzato come disposizione in serie di Full Adder è la **lentezza, dovuta alla catena dei riporti**. Questa realizzazione viene anche detta **ripple-carry adder**.



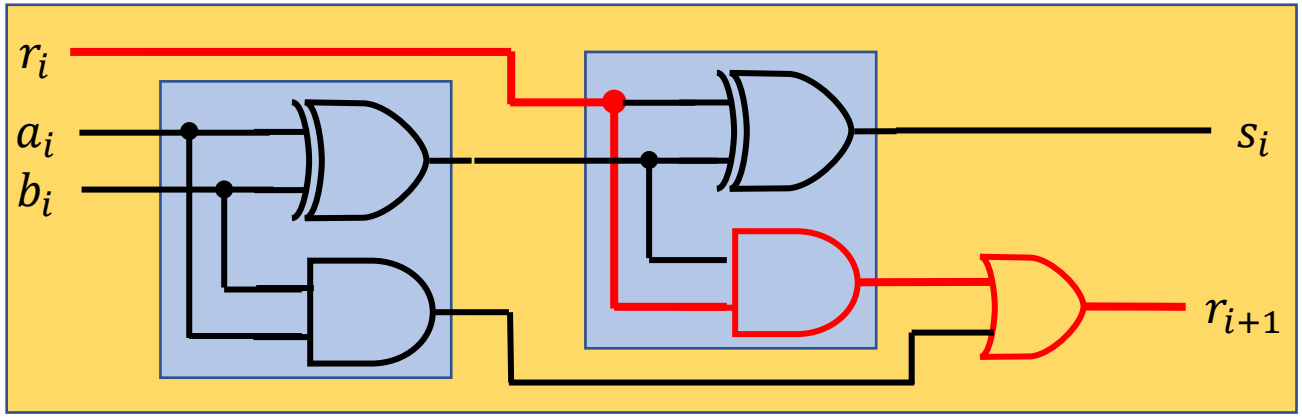
n-bit Adder



Adder a $3n$ bit

Ritardo FA

- Ogni FA infatti impiega 2 τ_p per portare a regime il riporto in uscita **da quando è a regime il riporto in ingresso.**
- Se eseguo operazioni tra dati a 64 bit, nel caso peggiore ho quindi il risultato aggiornato dopo 128 τ_p.



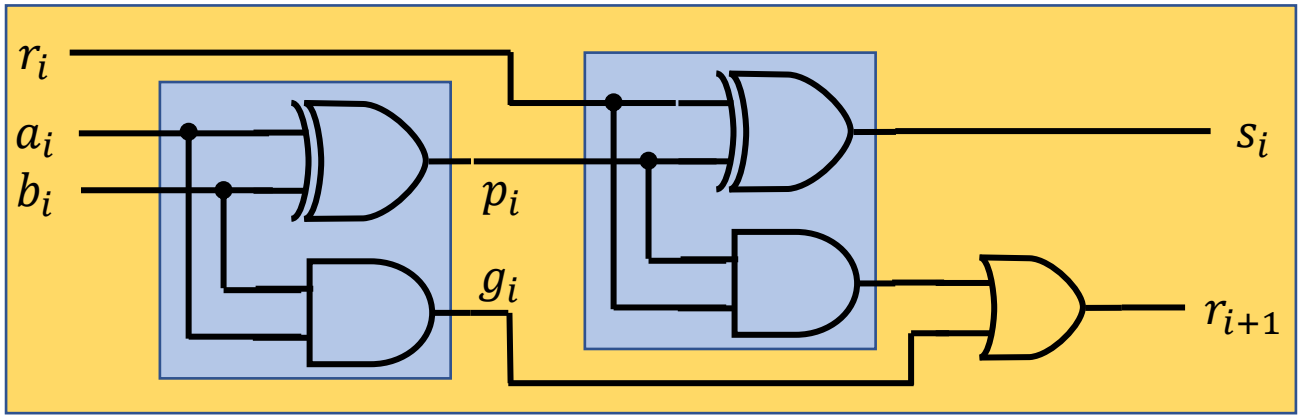
Carry lookahead

Una rete più veloce, ma più costosa dal punto di vista del numero di gate, si ottiene sintetizzando un circuito di **carry lookahead** che elabori gli output del primo half-adder. Definiamo, per ogni full-adder, i due segnali (corrispondenti alle uscite del primo HA)

carry propagate $p_i = a_i \oplus b_i$

carry generate $g_i = a_i b_i$

Questi segnali sono a regime dopo τ_p **per tutti i full-adder.**



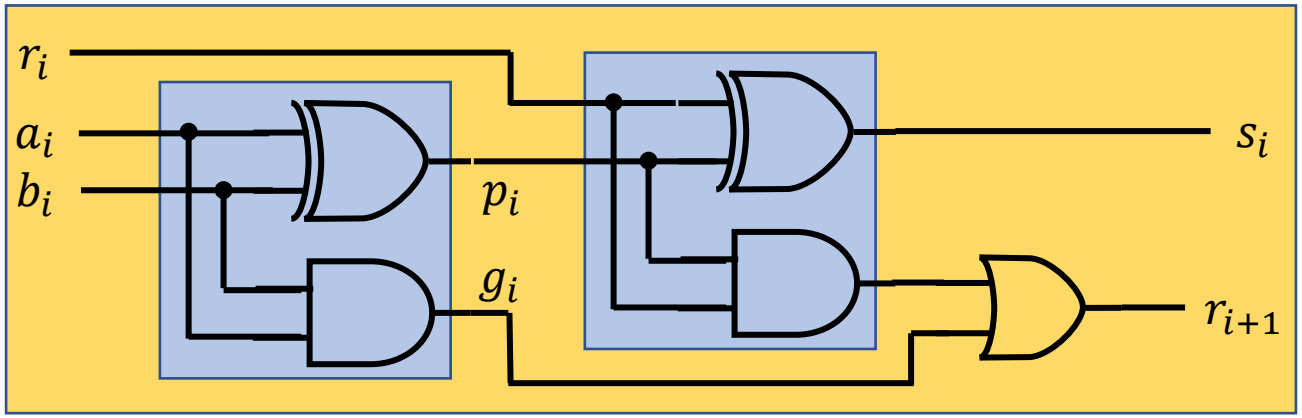
Carry lookahead

Le uscite del full-adder possono essere espresse in funzione di questi due segnali come

Bit di somma $s_i = r_i \oplus p_i$

Riporto $r_{i+1} = r_i\, p_i + g_i$

La seconda espressione identifica **una relazione ricorsiva** tra i riporti, esplicitata fino a $i = 3$ nella prossima slide



Carry Lookahead

$$r_0 = c_{in}$$

$$r_1 = g_0 + p_0 r_0$$

$$r_2 = g_1 + p_1 r_1 = g_1 + p_1 \left(g_0 + p_0 r_0\right)$$
$$= g_1 + p_1 g_0 + p_1 p_0 r_0$$

$$r_3 = g_2 + p_2 r_2 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 r_0$$

Carry Lookahead

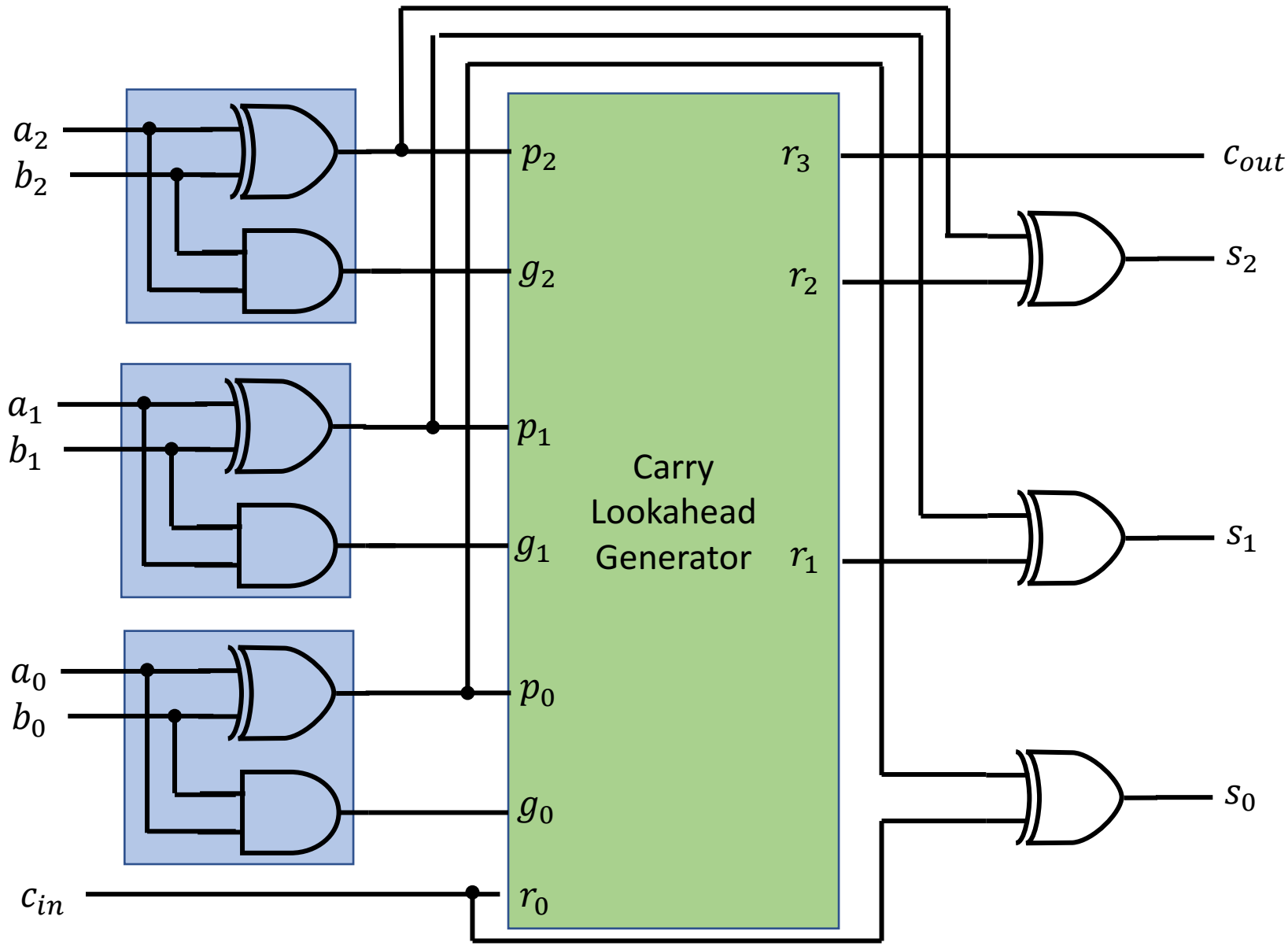
In generale, il riporto $i+1$ è dato da

$$r_{i+1} = \sum_{j=0}^{i} \left[g_j \prod_{k=j+1}^{i} p_k \right] + r_0 \prod_{j=0}^{i} p_j$$

ovvero è calcolabile,

- a partire da segnali a regime dopo τ_p da quando sono disponibili gli ingressi dell'adder (uscite HA)
- **con un circuito SP.**

Complessivamente, **ogni riporto è quindi disponibile** in teoria **dopo $3\tau_p$, indipendentemente dal numero di bit dell'adder**

Serve poi l'EX-OR tra il carry propagate p_i e il riporto r_i per calcolare **i bit di uscita s_i, che quindi sono pronti dopo $4\tau_p$**

3-bit Adder con carry lookahead